Data Analysis in Python Lab (2)

Part 5: Line Fitting

```
# imports the libraries needed to import data from google drive, read it in and plot it.
# grant permission to access your google drive.from google.colab import drivefrom google.colab import drive
from google.colab import drive
drive.mount('/content/drive')
import matplotlib.pyplot as plt
import math
import pandas
import numpy
import statsmodels.api as sm
```

```
Mounted at /content/drive
```

```
# read data
data = pandas.read_csv("/content/drive/MyDrive/Python/Data/linedata.csv")
data.head()
```

	x	y	yerr
0	1	3.21	0.35
1	2	3.40	0.40
2	3	4.63	0.45
3	4	3.97	0.50
4	5	3.21	0.55

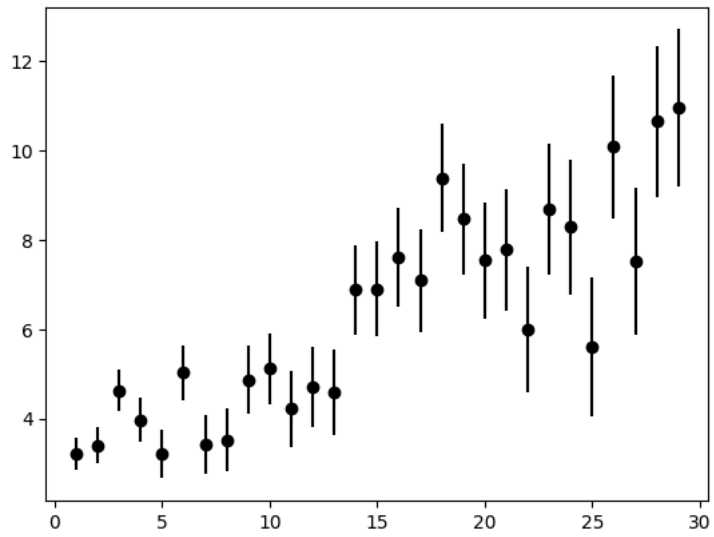
Next steps: Generate code with data | New interactive sheet

```
#  read data
data = pandas.read_csv("/content/drive/MyDrive/Python/Data/linedata.csv")  # Edit the name here to your data file.
data.head()

# pulling out the column containing the x-values, the y-values and the uncertainties in the y-values.

x = data['x']
y = data['y']
err_y = data['yerr']

# plotting the data
plt.errorbar(x,y, yerr=err_y, fmt="ko")
plt.show()
```




```
x1 = sm.add_constant(x)   # tells the package that you want an intercept, not a line through zero.
w = 1.0/(err_y**2)   # calculates how much weight to give each point. larger errors mean lower weights
model = sm.WLS(y,x1,weights=w) # generates model
result = model.fit() # generates fit
```

```
result.summary() # prints out lots of statistics of the fit


# it will print out a whole bunch of statistics. The ones you are mainly interested in are in the middle table.
# const is the intercept - the "coef" is its value and the "std err" is the standard deviation in this value.
# the line below (called x if your data column was called x) gives the slope - the "coef" is its value and the "std err" is
```

WLS Regression Results

Dep. Variable:	y	**R-squared:**	0.771
Model:	WLS	**Adj. R-squared:**	0.762
Method:	Least Squares	**F-statistic:**	90.78
Date:	Thu, 02 Apr 2026	**Prob (F-statistic):**	3.96e-10
Time:	02:01:54	**Log-Likelihood:**	-42.661
No. Observations:	29	**AIC:**	89.32
Df Residuals:	27	**BIC:**	92.06
Df Model:	1		
Covariance Type:	nonrobust		

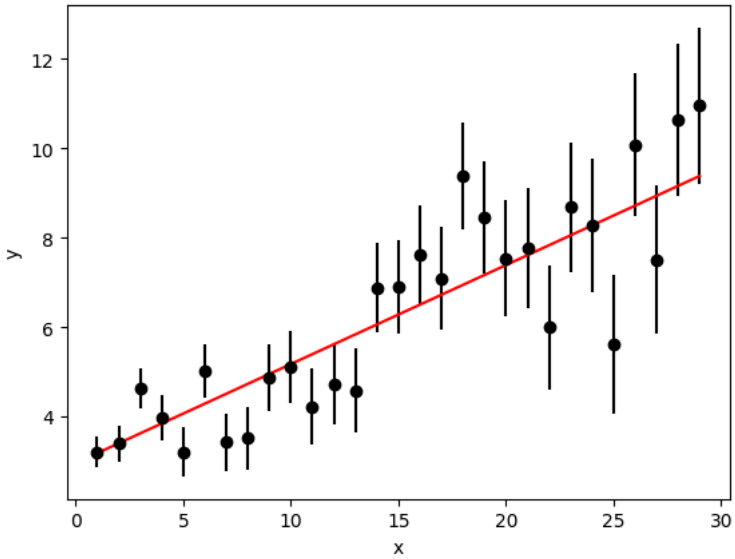
	coef	std err	t	P>|t|	[0.025	0.975]
const	2.9569	0.238	12.416	0.000	2.468	3.446
x	0.2216	0.023	9.528	0.000	0.174	0.269

Omnibus:	0.871	**Durbin-Watson:**	1.846
Prob(Omnibus):	0.647	**Jarque-Bera (JB):**	0.768
Skew:	-0.064	**Prob(JB):**	0.681
Kurtosis:	2.213	**Cond. No.**	15.2

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
plt.errorbar(x,y, yerr=err_y, fmt="ko") #plots the data points with error bars
bm = result.params[0]       #extracts the intercept (bm) and gradient (am) from the fit results
am = result.params[1]
model = am*x+bm             #calculates the y-values of the best fit line using y=ax+b at each x value
plt.plot(x,model,"-r")      #plots the best fit line in red (r), as a solid line
plt.xlabel("x")  #can change the name of the axes as needed
plt.ylabel("y")
plt.savefig("/content/drive/MyDrive/Python/Data/Myplot.png", dpi = 300) #save plot as png in drive
plt.show()  # displays plot
```

```
/tmp/ipykernel_2550/1478733293.py:2: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future
  bm = result.params[0]       #extracts the intercept (bm) and gradient (am) from the fit results
/tmp/ipykernel_2550/1478733293.py:3: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future
  am = result.params[1]
```



from result.summary(),

gradient = 0.2216 ± 0.023

intercept = 2.9569 ± 0.238